SI-2 PROIEKTUA - Errefaktorizazioa

Iago Vazquez, Pablo Martinez

Gaien Aurkibidea

1 Egilea: Pablo Martinez **2**
1.1 Write short units of code . . . 2
1.1.1 Hasierako kodea . . . 2
1.1.2 Errefaktorizatutako kodea . . . 3
1.1.3 Egindako errefaktorizazioaren deskribapena . . . 4
1.1.4 Egindako errefaktorizazioaren deskribapena . . . 4
1.2 Write simple units of code . . . 4
1.2.1 Hasierako kodea . . . 4
1.2.2 Errefaktorizatutako kodea . . . 5
1.2.3 Egindako errefaktorizazioaren deskribapena . . . 6
1.3 Duplicate code . . . 7
1.3.1 Hasierako kodea . . . 7
1.3.2 Errefaktorizatutako kodea . . . 7
1.3.3 Egindako errefaktorizazioaren deskribapena . . . 8
1.4 Keep unit interfaces small . . . 8
1.4.1 Hasierako kodea . . . 8
1.4.2 Errefaktorizatutako kodea . . . 9
1.4.3 Egindako errefaktorizazioaren deskribapena . . . 10

2 Egilea: Iago Vazquez **11**
2.1 Write short units of code . . . 11
2.1.1 Hasierako kodea . . . 11
2.1.2 Errefaktorizatutako kodea . . . 12
2.1.3 Egindako errefaktorizazioaren deskribapena . . . 13
2.2 Write simple units of code . . . 13
2.2.1 Hasierako kodea . . . 13
2.2.2 Errefaktorizatutako kodea . . . 14
2.2.3 Egindako errefaktorizazioaren deskribapena . . . 16
2.3 Duplicate code . . . 16
2.3.1 Hasierako kodea . . . 16
2.3.2 Errefaktorizatutako kodea . . . 16
2.3.3 Egindako errefaktorizazioaren deskribapena . . . 17
2.4 Keep unit interfaces small . . . 17
2.4.1 Hasierako kodea . . . 17
2.4.2 Errefaktorizatutako kodea . . . 17
2.4.3 Egindako errefaktorizazioaren deskribapena . . . 18

1 Egilea: Pablo Martinez

1.1 Write short units of code

1.1.1 Hasierako kodea

```
1     public boolean buyProduct(String buyerEmail, Integer saleNumber
          ) {
2         try {
3             db.getTransaction().begin();
4             Seller buyer = db.find(Seller.class, buyerEmail);
5             Sale sale = db.find(Sale.class, saleNumber);
6             Seller seller = sale.getSeller();
7 
8             if (buyer == null || sale == null || sale.getBuyer() !=
                  null) {
9                 db.getTransaction().rollback();
10                return false;
11            }
12 
13            float price = sale.getPrice();
14            if (buyer.getMoney() < price) {
15                db.getTransaction().rollback();
16                return false;
17            }
18 
19            buyer.addMoney(-price);
20            sale.setBuyer(buyer);
21            buyer.addPurchasedSale(sale);
22 
23            Bidalketa bidalketa = new Bidalketa(sale);
24            sale.setBidalketa(bidalketa);
25 
26            db.persist(bidalketa);
27 
28            // --- NUEVO BLOQUE MUGIMENDUAK ---
29            Mugimenduak mugimendu = new Mugimenduak("EROSKETA", new
                  java.util.Date(), buyer);
30            mugimendu.setSale(sale);
31            db.persist(mugimendu);
32            // --------------------------------
33 
34            Sale managedsale =db.merge(sale);
35            Seller manageSeller=db.merge(buyer);
36 
37            db.getTransaction().commit();
38            return true;
39        } catch (Exception e) {
40            e.printStackTrace();
41            db.getTransaction().rollback();
42            return false;
43        }
```

```
44      }
```

1.1.2 Errefaktorizatutako kodea

```
1       public boolean buyProduct(String buyerEmail, Integer saleNumber
            ) {
2           try {
3               db.getTransaction().begin();
4               Seller buyer = db.find(Seller.class, buyerEmail);
5               Sale sale = db.find(Sale.class, saleNumber);
6
7               if (!erosketaBideragarriaDa(buyer, sale)) {
8                   db.getTransaction().rollback();
9                   return false;
10              }
11
12              gauzatuErosketa(buyer, sale, sale.getPrice());
13
14              db.getTransaction().commit();
15              return true;
16          } catch (Exception e) {
17              e.printStackTrace();
18              db.getTransaction().rollback();
19              return false;
20          }
21      }
22
23      private boolean erosketaBideragarriaDa(Seller buyer, Sale sale)
            {
24          if (buyer == null || sale == null || sale.getBuyer() !=
               null) {
25              return false;
26          }
27          else if (buyer.getMoney() < sale.getPrice()) {
28              return false;
29          }
30          return true;
31      }
32
33      private void gauzatuErosketa(Seller buyer, Sale sale, float
            price) {
34          buyer.addMoney(-price);
35          sale.setBuyer(buyer);
36          buyer.addPurchasedSale(sale);
37
38          Bidalketa bidalketa = new Bidalketa(sale);
39          sale.setBidalketa(bidalketa);
40
41          db.persist(bidalketa);
42
```

```
43          Mugimenduak mugimendu = new Mugimenduak("EROSKETA", new
               java.util.Date(), buyer);
44          mugimendu.setSale(sale);
45          db.persist(mugimendu);
46
47          db.merge(sale);
48          db.merge(buyer);
49      }
```

1.1.3 Egindako errefaktorizazioaren deskribapena

1.1.4 Egindako errefaktorizazioaren deskribapena

Hasieran, `buyProduct` metodoak 15 lerro baino gehiago zituen. Programazio-praktika onen gomendioei jarraituz, metodo bakoitzak 15 lerro edo gutxiago izan beharko lituzke. Helburu hori lortzeko, jatorrizko metodoa bitan zatitu da. Hau garatzeko, Eclipse-k eskaintzen duen *Extract Method* tresna erabili da.

- **erosketaBideragarriaDa**: Metodo pribatu berri honek `boolean` bat bueltatzen du, zeinak erosketa bideragarria den ala ez adierazten duen. Horretarako, hasierako metodoan egiten ziren egiaztapen guztiak gauzatzen ditu.
- **gauzatuErosketa**: Datu-baseko eragiketak, entitateen eguneraketak eta `Bidalketa` zein `Mugimenduak` objektuen sorrera metodo honetan isolatu dira.

1.2 Write simple units of code

1.2.1 Hasierako kodea

```
1  public boolean addToBasket(String buyerEmail, Integer saleNumber) {
2      try {
3          db.getTransaction().begin();
4          Seller buyer = db.find(Seller.class, buyerEmail);
5          Sale sale = db.find(Sale.class, saleNumber);
6
7          if (buyer == null || sale == null || sale.getBuyer() !=
               null) {
8              db.getTransaction().rollback();
9              return false;
10         }
11
12         List<Sale> basket = buyer.getBasket();
13
14         for (Sale s : basket) {
15             if (s.getSaleNumber().equals(saleNumber)) {
16                 db.getTransaction().rollback();
17                 return false;
18             }
19         }
20
21         if (basket.isEmpty()) {
```

```
22              basket.add(sale);
23              db.merge(buyer);
24              db.getTransaction().commit();
25              return true;
26          }
27
28          Seller firstSeller = buyer.getBasket().get(0).getSeller();
29
30          if (!sale.getSeller().equals(firstSeller)) {
31              return false;
32          }
33
34          buyer.addToBasket(sale);
35          db.merge(buyer);
36
37          db.getTransaction().commit();
38          return true;
39
40      } catch (Exception e) {
41          e.printStackTrace();
42          db.getTransaction().rollback();
43          return false;
44      }
45  }
```

1.2.2 Errefaktorizatutako kodea

```
1       public boolean addToBasket(String buyerEmail, Integer
            saleNumber) {
2           try {
3               db.getTransaction().begin();
4               Seller buyer = db.find(Seller.class, buyerEmail);
5               Sale sale = db.find(Sale.class, saleNumber);
6
7               if (datuakFaltaDira(buyer, sale)) {
8                   db.getTransaction().rollback(); return false;
9               }
10
11              List<Sale> basket = buyer.getBasket();
12
13              if (produktuaSaskianDago(saleNumber, basket)) {
14                  db.getTransaction().rollback(); return false;
15              }
16
17              if (saltzaileEzberdinaDa(sale, basket)) {
18                  db.getTransaction().rollback(); return false;
19              }
20
21              buyer.addToBasket(sale);
22              db.merge(buyer);
23              db.getTransaction().commit();
```

```
24              return true;
25
26          } catch (Exception e) {
27              e.printStackTrace();
28              db.getTransaction().rollback();
29              return false;
30          }
31      }
32
33      private boolean datuakFaltaDira(Seller buyer, Sale sale) {
34          if (buyer == null || sale == null || sale.getBuyer() !=
                null) {
35              return true;
36          }
37          return false;
38      }
39
40      private boolean produktuaSaskianDago(Integer saleNumber, List<
            Sale> basket) {
41          for (Sale s : basket) {
42              if (s.getSaleNumber().equals(saleNumber)) {
43                  return true;
44              }
45          }
46          return false;
47      }
48
49      private boolean saltzaileEzberdinaDa(Sale sale, List<Sale>
            basket) {
50          if (!basket.isEmpty()) {
51              Seller firstSeller = basket.get(0).getSeller();
52              if (!sale.getSeller().equals(firstSeller)) {
53                  return true;
54              }
55          }
56          return false;
57      }
```

1.2.3 Egindako errefaktorizazioaren deskribapena

Hasieran `addToBasket` metodoak zrtziko konplexutasun ziklomatiko zuen. Programazio gomendioak jarraituz, metodo baten konplexutasunak 4 edo txikiagoa izan beharko luke. Helburu hau lortzeko, konpondu nahi den metodoa hainbat zatitan banatu da. Hau garatzeko, Eclipse-k eskainitako *Extract Method* tresna erabili da, barruko `if` eta `for` blokeak beste metodo pribatuetara ateraz.

- **datuakFaltaDira**: Metodo honek `boolean` bat bueltatzen du. Horretarako, hasierako metodoan egiten ziren lehenengo egiaztapenak gauzatzen ditu, hau da, `null` balioak bilatzen ditu `true` bueltatzeko.

- **produktuaSaskianDago**: Metodo honek `boolean` bat bueltatzen du. Bertan produktua jada saskian dagoen edo ez adierazten duen.
- **saltzaileEzberdinaDa**: Metodo honek `boolean` bat bueltatzen du. Saskiko lehen saltzailea eta produktu berriaren saltzailea bat ez datozen egiaztatzen du.

1.3 Duplicate code

1.3.1 Hasierako kodea

```
1     @WebMethod public Seller isLogged(String log, String pass) {
2          TypedQuery<Seller> query = db.createQuery("SELECT␣s␣FROM␣
              Seller␣s␣WHERE␣s.name=?1␣AND␣s.pass=?2",Seller.class);
3          query.setParameter(1, log);
4          query.setParameter(2, pass);
5          if(!query.getResultList().isEmpty()) //User Exististzen
              bada
6              return query.getResultList().get(0);
7          else return null;
8      }
9
10    public Admin isAdmin(String log, String pass) {
11        TypedQuery<Admin> query = db.createQuery(
12            "SELECT␣a␣FROM␣Admin␣a␣WHERE␣a.name=?1␣AND␣a.pass=?2",
13            Admin.class
14        );
15        query.setParameter(1, log);
16        query.setParameter(2, pass);
17
18        if (!query.getResultList().isEmpty())
19            return query.getResultList().get(0);
20        else
21            return null;
22    }
```

1.3.2 Errefaktorizatutako kodea

```
1     private <T> T erabiltzaileaBilatu(String mezua, Class<T> mota,
         String log, String pass) {
2          TypedQuery<T> query = db.createQuery(mezua, mota);
3          query.setParameter(1, log);
4          query.setParameter(2, pass);
5
6          if (!query.getResultList().isEmpty())
7              return query.getResultList().get(0);
8          else
9              return null;
10    }
11
12     @WebMethod public Seller isLogged(String log, String pass) {
```

```
13          return erabiltzaileaBilatu("SELECT␣s␣FROM␣Seller␣s␣WHERE␣s.
               name=?1␣AND␣s.pass=?2", Seller.class, log, pass);
14      }
15
16      public Admin isAdmin(String log, String pass) {
17          return erabiltzaileaBilatu("SELECT␣a␣FROM␣Admin␣a␣WHERE␣a.
               name=?1␣AND␣a.pass=?2", Admin.class, log, pass);
18      }
```

1.3.3 Egindako errefaktorizazioaren deskribapena

Hasieran `isLogged` eta `isAdmin` metodoek kode bikoiztua zuten, baina bakoitzak klase desberdin batekin lan egiten zuen (`Seller` eta `Admin`). Programazio gomendioak jarraituz, kode errepikatua saihestu behar da. Helburu hau lortzeko, kode amankomun hori atera da eta metodo pribatu laguntzaile bat sortu da. Aldaketa hauen ondorioz, kode errepikakorra ezabatu eta behin bakarrik definituta izatea lortu da.

1.4 Keep unit interfaces small

1.4.1 Hasierako kodea

```
1       public Sale createSale(String title, String description, int
           status, float price, Date pubDate, String sellerEmail, File
           file) throws FileNotUploadedException,
           MustBeLaterThanTodayException, SaleAlreadyExistException {
2           System.out.println(">>␣DataAccess:␣createProduct=>␣title=␣"
               +title+"␣seller="+sellerEmail);
3           try {
4               if(pubDate.before(UtilDate.trim(new Date()))) {
5                   throw new MustBeLaterThanTodayException(
                       ResourceBundle.getBundle(EMAITZA).getString("
                       DataAccess.ErrorSaleMustBeLaterThanToday"));
6               }
7               if (file==null)
8                   throw new FileNotUploadedException(ResourceBundle.
                       getBundle(EMAITZA).getString("DataAccess.
                       ErrorFileNotUploadedException"));
9
10              db.getTransaction().begin();
11
12              Seller seller = db.find(Seller.class, sellerEmail);
13              if (seller.doesSaleExist(title)) {
14                  db.getTransaction().commit();
15                  throw new SaleAlreadyExistException(ResourceBundle.
                       getBundle(EMAITZA).getString("DataAccess.
                       SaleAlreadyExist"));
16              }
17
18              Sale sale = seller.addSale(title, description, status,
                   price, pubDate, file);
```

```
19
20              db.persist(seller);
21              db.getTransaction().commit();
22              System.out.println("sale␣stored␣"+sale+ "␣"+seller);
23
24              System.out.println("hasta␣aqui");
25              return sale;
26          } catch (NullPointerException e) {
27                  java.util.logging.Logger.getLogger(DataAccess.class
                       .getName()).log(java.util.logging.Level.SEVERE,
                       "NullPointerException␣createSale␣metodoan", e);
                                  db.getTransaction().commit();
28              return null;
29          }
30      }
```

1.4.2 Errefaktorizatutako kodea

```
1       public Sale createSale(String sellerEmail, File file, Sale
           newSale) throws FileNotUploadedException,
           MustBeLaterThanTodayException, SaleAlreadyExistException {
2            System.out.println(">>␣DataAccess:␣createProduct=>␣title=␣"
                +newSale.getTitle()+"␣seller="+sellerEmail);
3            try {
4                if(newSale.getPubDate().before(UtilDate.trim(new Date()
                    ))) {
5                    throw new MustBeLaterThanTodayException(
                        ResourceBundle.getBundle(EMAITZA).getString("
                        DataAccess.ErrorSaleMustBeLaterThanToday"));
6                }
7                if (file==null)
8                    throw new FileNotUploadedException(ResourceBundle.
                        getBundle(EMAITZA).getString("DataAccess.
                        ErrorFileNotUploadedException"));
9
10               db.getTransaction().begin();
11
12               Seller seller = db.find(Seller.class, sellerEmail);
13               if (seller.doesSaleExist(newSale.getTitle())) {
14                   db.getTransaction().commit();
15                   throw new SaleAlreadyExistException(ResourceBundle.
                        getBundle(EMAITZA).getString("DataAccess.
                        SaleAlreadyExist"));
16               }
17
18               Sale sale = seller.addSale(newSale, file);
19
20               db.persist(seller);
21               db.getTransaction().commit();
22               System.out.println("sale␣stored␣"+sale+ "␣"+seller);
23
```

```
24              System.out.println("hasta␣aqui");
25              return sale;
26          } catch (NullPointerException e) {
27                  java.util.logging.Logger.getLogger(DataAccess.class
                        .getName()).log(java.util.logging.Level.SEVERE,
                        "NullPointerException␣createSale␣metodoan", e);
                                db.getTransaction().commit();
28              return null;
29          }
30      }
```

1.4.3 Egindako errefaktorizazioaren deskribapena

Hasieran, `createSale` metodoak (eta datu-baseko zein domeinuko metodo baliokideek) 7 parametro zituen, "Small Interfaces"araua argi eta garbi urratuz. Programazio-gomendioei jarraituz, metodo baten parametro kopuruak gehienez 4 izan behar du, irakurgarritasuna eta mantengarritasuna hobetzeko.

Arazo hau konpontzeko, *Introduce Parameter Object* errefaktorizazio-teknika aplikatu da. Zuzeneko propietate solte guztiak (`Sale` baten berezko datuak direnak) `Sale` motako objektu bakar batean bildu dira, geruza ezberdinetan zehar informazioa garraiatzeko erabiliz. Era berean, `Seller` klaseko `addSale` metodoa ere eguneratu da parametro berdinak jasotzeko.

Geruzetan oinarritutako arkitektura denez, `DataAccess` klasean egindako aldaketa honek proiektu osoko dependentziak eguneratzea esan nahi izan du: `BLFacade` interfazea, GUI-ko deiak eta Mockito bidezko testak moldatu dira `Sale` objektu berriarekin lan egiteko. Aldaketa hauen ondorioz, metodo nagusiek 3 parametro soilik jasotzen dituzte orain (`Sale` objektua, saltzailearen emaila eta irudiaren fitxategia). Interfazea nabarmen txikitu da, 4ko muga ezin hobeto betez, eta etorkizunean produktuaren ezaugarri berri bat gehituko balitz, ez litzateke metodo hauen sinadura aldatu beharko.

2 Egilea: Iago Vazquez

2.1 Write short units of code

2.1.1 Hasierako kodea

```
1     public boolean buyBasket(String buyerEmail) {
2         try {
3             db.getTransaction().begin();
4             Seller buyer = db.find(Seller.class, buyerEmail);
5             if (buyer == null) {
6                 db.getTransaction().rollback();
7                 return false;
8             }
9             List<Sale> basket = buyer.getBasket();
10            if (basket == null || basket.isEmpty()) {
11                db.getTransaction().rollback();
12                return false;
13            }
14            Seller seller = basket.get(0).getSeller();
15            float total = 0;
16            for (Sale s : basket) {
17                if (!s.getSeller().equals(seller)) {
18                    db.getTransaction().rollback();
19                    return false;
20                }
21                total += s.getPrice();
22            }
23            if (buyer.getMoney() < total) {
24                db.getTransaction().rollback();
25                return false;
26            }
27            buyer.addMoney(-total);
28            for (Sale s : basket) {
29                s.setBuyer(buyer);
30                buyer.addPurchasedSale(s);
31                Bidalketa bidalketa = new Bidalketa(s);
32                s.setBidalketa(bidalketa);
33                db.persist(bidalketa);
34                Mugimenduak mugimendu = new Mugimenduak("EROSKETA",
                      new java.util.Date(), buyer);
35                mugimendu.setSale(s);
36                db.persist(mugimendu);
37                db.merge(s);
38            }
39            basket.clear();
40            db.merge(buyer);
41            db.merge(seller);
42            db.getTransaction().commit();
43            return true;
44        } catch (Exception e) {
45            e.printStackTrace();
```

```
46              if (db.getTransaction().isActive()) {
47                  db.getTransaction().rollback();
48              }
49              return false;
50          }
51      }
```

2.1.2 Errefaktorizatutako kodea

```
1       public boolean buyBasket(String buyerEmail) {
2           try {
3               db.getTransaction().begin();
4               Seller buyer = db.find(Seller.class, buyerEmail);
5
6               if (buyer == null || buyer.getBasket() == null || buyer
                    .getBasket().isEmpty()) {
7                   db.getTransaction().rollback();
8                   return false;
9               }
10
11              float total = saskiarenTotalaKalkulatu(buyer.getBasket
                    ());
12              if (total < 0 || buyer.getMoney() < total) {
13                  db.getTransaction().rollback();
14                  return false;
15              }
16
17              saskiaErosi(buyer, total);
18
19              db.getTransaction().commit();
20              return true;
21          } catch (Exception e) {
22              e.printStackTrace();
23              if (db.getTransaction().isActive()) {
24                  db.getTransaction().rollback();
25              }
26              return false;
27          }
28      }
29
30      private float saskiarenTotalaKalkulatu(List<Sale> basket) {
31          Seller seller = basket.get(0).getSeller();
32          float total = 0;
33          for (Sale s : basket) {
34              if (!s.getSeller().equals(seller)) return -1;
35              total += s.getPrice();
36          }
37          return total;
38      }
39
40      private void saskiaErosi(Seller buyer, float total) {
```

```
41          buyer.addMoney(-total);
42          for (Sale s : buyer.getBasket()) {
43              s.setBuyer(buyer);
44              buyer.addPurchasedSale(s);
45
46              Bidalketa bidalketa = new Bidalketa(s);
47              s.setBidalketa(bidalketa);
48              db.persist(bidalketa);
49
50              Mugimenduak mugimendu = new Mugimenduak("EROSKETA", new
                    java.util.Date(), buyer);
51              mugimendu.setSale(s);
52              db.persist(mugimendu);
53              db.merge(s);
54          }
55          buyer.getBasket().clear();
56          db.merge(buyer);
57          db.merge(buyer.getBasket().get(0).getSeller());
58      }
```

2.1.3 Egindako errefaktorizazioaren deskribapena

Hasieran, buyBasket metodoak 15 lerro baino askoz gehiago zituen (ia 50). Mantengarritasuna eta irakurgarritasuna hobetzeko, **Extract Method errefaktorizazio-teknika** erabili da. Totala kalkulatzeko logika **saskiarenTotalaKalkulatu** metodora mugitu da, eta entitateak prozesatzeko logika **saskiaErosi** metodo pribatura atera da. Orain metodo bakoitza bere helburu espezifikoaz arduratzen da

2.2 Write simple units of code

2.2.1 Hasierako kodea

```
1       public boolean resolveErreklamazioa(Integer saleNumber, boolean
            accept) {
2           try {
3               db.getTransaction().begin();
4               Sale sale = db.find(Sale.class, saleNumber);
5               if (sale == null || sale.getErreklamazioa() == null) {
6                   db.getTransaction().rollback(); return false;
7               }
8
9               if (accept) {
10                  Seller buyer = sale.getBuyer();
11                  Seller seller = sale.getSeller();
12
13                  if (buyer != null && seller != null) {
14                      buyer.addMoney(sale.getPrice());
15                      seller.addMoney(-sale.getPrice());
16
17                      Mugimenduak mBuyer = new Mugimenduak("ITZULKETA
                            ", new java.util.Date(), buyer);
```

```
18                      mBuyer.setSale(sale);
19                      db.persist(mBuyer);
20
21                      Mugimenduak mSeller = new Mugimenduak("
                            ITZULKETA_KENTZEA", new java.util.Date(),
                            seller);
22                      mSeller.setSale(sale);
23                      db.persist(mSeller);
24
25                      buyer.getPurchasedSales().remove(sale);
26                      sale.setBuyer(null);
27
28                      db.merge(buyer);
29                      db.merge(seller);
30                  }
31              }
32
33              Erreklamazioa err = sale.getErreklamazioa();
34              sale.setErreklamazioa(null);
35              db.merge(sale);
36
37              if (err != null) {
38                  Erreklamazioa dbErr = db.find(Erreklamazioa.class,
                        err.getId());
39                  if (dbErr != null) err.setTratatuta(true);
40                  db.merge(err);
41              }
42
43              db.getTransaction().commit();
44              return true;
45          } catch (Exception e) {
46              // ...
47          }
48      }
```

2.2.2 Errefaktorizatutako kodea

```
1       public boolean resolveErreklamazioa(Integer saleNumber, boolean
            accept) {
2           try {
3               db.getTransaction().begin();
4               Sale sale = db.find(Sale.class, saleNumber);
5               if (sale == null || sale.getErreklamazioa() == null) {
6                   db.getTransaction().rollback(); return false;
7               }
8
9               if (accept) {
10                  diruaEtaMugimenduakItzuli(sale);
11              }
12
13              erreklamazioaItxi(sale);
```

```
14
15              db.getTransaction().commit();
16              return true;
17          } catch (Exception e) {
18              e.printStackTrace();
19              db.getTransaction().rollback();
20              return false;
21          }
22      }
23
24          private void erreklamazioaItxi(Sale sale) {
25                  Erreklamazioa err = sale.getErreklamazioa();
26                  sale.setErreklamazioa(null);
27                  db.merge(sale);
28
29                  if (err != null) {
30                      Erreklamazioa dbErr = db.find(Erreklamazioa.
                            class, err.getId());
31                      if (dbErr != null) err.setTratatuta(true);
32                      db.merge(err);
33                  }
34          }
35
36          private void diruaEtaMugimenduakItzuli(Sale sale) {
37                  Seller buyer = sale.getBuyer();
38                  Seller seller = sale.getSeller();
39
40                  if (buyer != null && seller != null) {
41                      buyer.addMoney(sale.getPrice());
42                      seller.addMoney(-sale.getPrice());
43
44
45                      Mugimenduak mBuyer = new Mugimenduak("ITZULKETA
                            ", new java.util.Date(), buyer);
46                      mBuyer.setSale(sale);
47                      db.persist(mBuyer);
48
49                      Mugimenduak mSeller = new Mugimenduak("
                            ITZULKETA_KENTZEA", new java.util.Date(),
                            seller);
50                      mSeller.setSale(sale);
51                      db.persist(mSeller);
52
53
54                      buyer.getPurchasedSales().remove(sale);
55                      sale.setBuyer(null);
56
57                      db.merge(buyer);
58                      db.merge(seller);
59                  }
60          }
```

2.2.3 Egindako errefaktorizazioaren deskribapena

Hasieran `resolveErreklamazioa` metodoak 4 baino konplexutasun ziklomatiko altuagoa zuen, `if` baldintza eta adar barneratu ugari zituelako. Programazio-praktika onen gomendioei jarraituz, konplexutasuna muga onargarrian (4 edo gutxiago) izan dadin, **Eclipse-k eskainitako *Extract Method* tresna erabili da**.

Eskuzko akatsak saihestuz, barruko logika-blokeak aztertu eta bi metodo pribatu berritara atera dira automatikoki:

- `diruaEtaMugimenduakItzuli`: Erosleari dirua itzultzeko, saltzaileari kentzeko eta dagozkion mugimenduak erregistratzeko kodea isolatzen du.
- `erreklamazioaItxi`: Erreklamazioaren egoera datu-basean eguneratu eta tratatuta gisa markatzeko eragiketak kudeatzen ditu.

2.3 Duplicate code

2.3.1 Hasierako kodea

```
1  // getSales metodoan:
2          List<Sale> sales = query.getResultList();
3          for (Sale sale:sales){
4              res.add(sale);
5          }
6          return res;
7
8  // getPublishedSales metodoan:
9          List<Sale> sales = query.getResultList();
10         for (Sale sale:sales){
11             res.add(sale);
12         }
13         return res;
```

2.3.2 Errefaktorizatutako kodea

```
1      public List<Sale> getSales(String desc) {
2          TypedQuery<Sale> query = db.createQuery("SELECT␣s␣FROM␣Sale
               ␣s␣WHERE␣s.title␣LIKE␣?1␣AND␣s.buyer␣IS␣NULL", Sale.
               class);
3          query.setParameter(1, "%"+desc+"%");
4          return emaitzakLortu(query);
5      }
6
7      public List<Sale> getPublishedSales(String desc, Date pubDate)
           {
8          TypedQuery<Sale> query = db.createQuery("SELECT␣s␣FROM␣Sale
               ␣s␣WHERE␣s.title␣LIKE␣?1␣AND␣s.pubDate␣<=?2␣AND␣s.buyer␣
               IS␣NULL",Sale.class);
9          query.setParameter(1, "%"+desc+"%");
```

```
10          query.setParameter(2, pubDate);
11          return emaitzakLortu(query);
12      }
13
14      private List<Sale> emaitzakLortu(TypedQuery<Sale> query) {
15          List<Sale> res = new ArrayList<Sale>();
16          List<Sale> sales = query.getResultList();
17          for (Sale sale:sales){
18              res.add(sale);
19          }
20          return res;
21      }
```

2.3.3 Egindako errefaktorizazioaren deskribapena

getSales eta getPublishedSales itzulerako zerrenda lortzeko kode zehatz bera exekutatzen zuten. Datu horiek errepikatu beharrean, **emaitzakLortu** metodo pribatu laguntzaile bat sortu da, query bat parametro gisa jaso eta lortutako salmentak itzultzen dituena.

2.4 Keep unit interfaces small

2.4.1 Hasierako kodea

```
1  public boolean addEskaintza(Integer eskaeraId, String sellerEmail,
       float price, String message) {
2          // ...
3      }
```

2.4.2 Errefaktorizatutako kodea

```
1  public boolean addEskaintza(Eskaintza eskaintzaBerria) {
2          try {
3              db.getTransaction().begin();
4
5              Integer eskaeraId = eskaintzaBerria.getEskaera().getId
                   ();
6              String sellerEmail = eskaintzaBerria.getSeller().
                   getEmail();
7
8              Eskaera eskaera = db.find(Eskaera.class, eskaeraId);
9              Seller seller = db.find(Seller.class, sellerEmail);
10
11             if (eskaera == null || seller == null || eskaera.
                   isClosed() || eskaera.getBuyer().getEmail().equals(
                   sellerEmail)) {
12                 db.getTransaction().rollback();
13                 return false;
14             }
15
```

```
16              eskaintzaBerria.setSeller(seller);
17              eskaintzaBerria.setEskaera(eskaera);
18
19              eskaera.addEskaintza(eskaintzaBerria);
20
21              db.persist(eskaintzaBerria);
22              eskaera = db.merge(eskaera);
23
24              db.getTransaction().commit();
25              return true;
26          } catch (Exception e) {
27              e.printStackTrace();
28              db.getTransaction().rollback();
29              return false;
30          }
31      }
```

2.4.3 Egindako errefaktorizazioaren deskribapena

Hasieran, `addEskaintza` metodoak (eta datu-baseko zein domeinuko metodo baliokideek) 4 parametro solte zituen (`eskaeraId`, `sellerEmail`, `price`, `message`), "Small Interfaces"arauaren muga estutu edo lausotuz, nahiz eta gomendio estandarra 4 edo gutxiago izan, objektu-orientatutako diseinu egoki batek erlazionatutako datuak elkartzea eskatzen du mantengarritasuna hobetzeko.

Arazo hau konpontzeko, *Preserve Whole Object / Introduce Parameter Object* errefaktorizazio-teknika aplikatu da. Zuzeneko propietate solte guztiak (`Eskaintza` baten berezko datuak direnak) `Eskaintza` motako objektu bakar batean bildu dira, geruza ezberdinetan zehar informazioa garraiatzeko erabiliz. Era berean, klaseetako metodoak eguneratu dira parametro berdinak jasotzeko.

Geruzetan oinarritutako arkitektura denez, `DataAccess` klasean egindako aldaketa honek proiektu osoko dependentziak eguneratzea esan nahi izan du: `BLFacade` interfazea, GUI-ko deiak (`ViewEskaerakGUI`) eta JUnit bidezko testak (`acceptEskaintzaBDBlackTest` zein `WhiteTest`) moldatu dira `Eskaintza` objektu berriarekin lan egiteko. Aldaketa hauen ondorioz, metodo nagusiek parametro bakarra jasotzen dute orain (`Eskaintza` objektua bera). Interfazea nabarmen txikitu da, gehienezko muga ezin hobeto betez, eta etorkizunean eskaintzaren ezaugarri berri bat gehituko balitz, ez litzateke metodo hauen sinadura aldatu beharko.